

CO2 ASSESSMENT TOOL 3 DEBUGGING & OPTIMISATION TASK

Name:- Y.Thanushma

Reg. Number:- 192425355

Course Code:-CSA0617-Design and Algorithm Analysis

Q1. Debugging Maximum Element Search Code — Empty List and Initialization Risk

1. Problem Understanding & Context Analysis

Problem Statement

The given function is designed to find the largest element in a list using a linear search approach. The main problem is that it immediately accesses the first element using `arr[0]`.

This creates an assumption that the input list will always contain at least one element. If an empty list is supplied, the program produces an `IndexError`.

Purpose of the Program

- Find the maximum value in a list.
- Compare each element with the current maximum.
- Safely handle an empty list.
- Avoid accessing elements that do not exist.

Example Input:

`arr = [12, 7, 25, 18, 30]`

Expected Output:

Maximum Element: 30

Algorithm Concept Involved

- Linear Search
- Iteration
- Defensive Programming

- Edge-case handling

Location of the Error

```
max_val = arr[0]
```

Root Cause Analysis

The original code is:

```
def find_max(arr):  
  
    max_val = arr[0]  
  
    for i in range(1, len(arr)):  
  
        if arr[i] > max_val:  
  
            max_val = arr[i]  
  
    return max_val
```

The statement `arr[0]` is valid only when the list contains at least one element.

For:

```
arr = []
```

there is no index 0, so Python raises:

IndexError: list index out of range

Identified Issues

S.No	Issue	Type	Explanation
1	<code>arr[0]</code> used without validation	Runtime Error	Fails when the input list is empty
2	Assumes non-empty input	Robustness Issue	The function does not define behavior for an empty list
3	No input validation	Defensive Programming Issue	Invalid/edge-case input is not handled explicitly

2. Application of Concepts

Original Approach

The program initializes:

```
max_val = arr[0]
```

and then compares the remaining elements with it.

This approach is correct for a non-empty list, but it is unsafe when the list has no elements.

Corrected Approach

Before accessing the first element, check whether the list is empty:

```
if not arr:
```

```
    return None
```

Then initialize the maximum:

```
max_val = arr[0]
```

and search through the remaining elements.

Rules Applied

1. Validate the input before accessing an element.
2. Use the first valid element as the initial maximum.
3. Start the loop from index 1.
4. Update the maximum only when a larger value is found.
5. Return a clearly defined result for an empty list.

Why This Works

- For a non-empty list, `max_val` represents the largest value found so far.
- For an empty list, the function returns `None` immediately, so no invalid index is accessed.

3. Analytical & Decision-Making Skills

Debugging Process

Step 1: Identify the assumption

The code assumes:

The list contains at least one element.

Step 2: Test the edge case

```
find_max([])
```

This exposes the `arr[0]` problem.

Step 3: Locate the failure

```
max_val = arr[0]
```

fails because index 0 does not exist.

Step 4: Add defensive validation

```
if not arr:
```

```
    return None
```

Step 5: Test normal input

```
[12, 7, 25, 18, 30]
```

The result should be 30.

Step 6: Test edge input

```
[]
```

The result should be None.

Optimization Applied

- Empty input is checked before accessing the list.
- The first element is not compared with itself.
- The loop begins at index 1.
- Variable names are kept simple and meaningful.
- A docstring documents the function's behavior.

Why the Optimization is Useful

The main improvement is not a reduction in asymptotic complexity, because maximum search still requires examining the elements. Instead, it makes the function safer, clearer, and easier to maintain.

4. Solution Design & Approach

Corrected & Optimized Code

```
def find_max(arr):  
    """  
    Find the maximum element in a list.  
    Returns None if the list is empty.  
    """  
    if not arr:  
        return None  
    max_val = arr[0]  
    for value in arr[1:]:  
        if value > max_val:  
            max_val = value  
    return max_val  
  
arr = [10, 25, 5, 40, 15]  
print("Maximum Element:", find_max(arr))
```

Algorithm / Logic Flow

1. Start.
2. Receive the input list.
3. Check whether the list is empty.
4. If empty, return None.

5. Otherwise, assign the first element to max_val.
6. Traverse the remaining elements.
7. Compare each element with max_val.
8. Replace max_val if a larger element is found.
9. Return max_val.
10. Stop.

5. Clarity, Justification & Presentation

Time Complexity

Case	Time Complexity
Best Case	$O(n)$
Average Case	$O(n)$
Worst Case	$O(n)$

The algorithm examines each element at most once, so the running time is $O(n)$.

Space Complexity

$O(1)$

Only the variable max_val and a few loop variables are used, so no additional array is required.

Sample Input

arr = [12, 7, 25, 18, 30]

Expected Output

Maximum Element: 30

Final Conclusion

- The main defect is the direct use of arr[0] without checking whether the list contains an element.
- An empty-list check makes the function robust against this edge case.

- The first element is used as the initial maximum only after confirming that the list is non-empty.
- Starting the traversal from index 1 avoids redundant comparison.
- The corrected solution retains $O(n)$ time complexity and $O(1)$ space complexity while providing safer and clearer behavior.

Output:

```
File Edit Format Run Options Window Help
def find_max(arr):
    """
    Find the maximum element in a list.
    Returns None if the list is empty.
    """

    if not arr:
        return None

    max_val = arr[0]

    for value in arr[1:]:
        if value > max_val:
            max_val = value

    return max_val

arr = [10, 25, 5, 40, 15]

print("Maximum Element:", find_max(arr))
```

```
=====
Maximum Element: 40
|
```

Q2. Debugging Insertion Sort Code — Incorrect Final Placement Index

1. Problem Understanding & Context Analysis

Problem Statement

The given program uses Insertion Sort to arrange elements in ascending order. The shifting operation is correct, but the final statement places the key at the wrong index.

The error is:

```
arr[j] = key
```

After the shifting loop, j has moved one position before the correct insertion position.

Therefore, the key must be placed at:

```
arr[j + 1] = key
```

Purpose of the Program

- Sort the elements in ascending order.
- Maintain a sorted portion of the array.
- Shift larger elements to the right.
- Insert the current element into its correct position.

Example Input

arr = [5, 2, 4, 6, 1, 3]

Expected Output

[1, 2, 3, 4, 5, 6]

Algorithm Concept Involved

- Insertion Sort
- Element shifting
- Sorted-prefix maintenance

Location of the Error

arr[j] = key # ERROR

Root Cause Analysis

The original code is:

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j] = key
```

During the while loop, every element larger than key is shifted one position to the right. At the end of the loop, j has already been decreased.

Therefore, the correct position for key is $j + 1$, not j .

Identified Error

S.No	Error	Type of Error	Explanation
1	$\text{arr}[j] = \text{key}$	Indexing/Logical Error	Places key one position before its correct location
2	Wrong final insertion position	Algorithm Error	Can overwrite a value in the sorted prefix

2. Application of Concepts

Correct Insertion Logic

The shifting operation is:

while $j \geq 0$ and $\text{arr}[j] > \text{key}$:

$\text{arr}[j + 1] = \text{arr}[j]$

$j -= 1$

After this loop, j points to the position immediately before where key should be inserted.

Therefore:

$\text{arr}[j + 1] = \text{key}$

must be used.

Step-by-Step Example

Consider:

[5, 2, 4]

When $\text{key} = 2$:

$j = 0$

Since $5 > 2$, shift 5:

[5, 5, 4]

Then:

$$j = -1$$

The correct insertion position is:

$$j + 1 = 0$$

So:

[2, 5, 4]

If $\text{arr}[j] = \text{key}$ were used, it would attempt to use index -1, producing incorrect behavior.

Rules Followed

1. Select the current element as key.
2. Compare it with the sorted elements before it.
3. Shift larger elements one position right.
4. Decrease j .
5. Insert key at $j + 1$.

3. Analytical & Decision-Making Skills

Debugging Process

Step 1: Identify the shifting operation.

$$\text{arr}[j + 1] = \text{arr}[j]$$

Step 2: Observe that j is decreased after each shift.

$$j -= 1$$

Step 3: Determine the correct insertion position.

$$j + 1$$

Step 4: Replace the incorrect statement:

$$\text{arr}[j] = \text{key}$$

with:

`arr[j + 1] = key`

Step 5: Test the algorithm using an unsorted array.

[5, 2, 4, 6, 1, 3]

Optimization Applied

- Use meaningful variable names: i, j, and key.
- Start from index 1 because the first element is already considered sorted.
- Shift elements instead of repeatedly swapping them.
- Insert the key directly into its correct position.
- Use comments to clearly describe each operation.

Why the Correction is Accurate

At the end of the shifting process:

j = position immediately before the insertion location

Hence:

correct position = j + 1

This preserves the sorted prefix after every iteration.

4. Solution Design & Approach

Correct and Optimized Code

```
def insertion_sort(arr):  
    """  
    Sort the list in ascending order using Insertion Sort.  
    """  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1
```

```
    arr[j + 1] = key
    return arr
arr = [5, 2, 4, 6, 1, 3]
print("Sorted Array:", insertion_sort(arr))
```

Output

Sorted Array: [1, 2, 3, 4, 5, 6]

Algorithm / Logic Flow

1. Start.
2. Take the second element as key.
3. Compare key with elements on its left.
4. Shift larger elements one position to the right.
5. Move j backward.
6. Insert key at j + 1.
7. Repeat for all remaining elements.
8. Return the sorted array.
9. Stop.

5. Clarity, Justification & Presentation

Time Complexity Analysis

Case	Time Complexity	Reason
Best Case	$O(n)$	Array is already sorted
Average Case	$O(n^2)$	Elements require multiple shifts
Worst Case	$O(n^2)$	Array is in reverse order

Space Complexity

$O(1)$ — The sorting is performed in-place and requires only a few extra variables.

Sample Input

```
arr = [5, 2, 4, 6, 1, 3]
```

Expected Output

Sorted Array: [1, 2, 3, 4, 5, 6]

Final Conclusion

- The main error is the use of `arr[j] = key`.
- During shifting, `j` moves one position before the correct insertion location.
- The correct statement is `arr[j + 1] = key`.
- The corrected algorithm maintains the sorted prefix after every iteration.
- Insertion Sort has $O(n)$ best-case and $O(n^2)$ average/worst-case time complexity.
- Its space complexity is $O(1)$.

Output:

```
File Edit Format Run Options Window Help
def insertion_sort(arr):
    """
    Sort the list in ascending order using Insertion Sort.
    """

    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr
arr = [5, 2, 4, 6, 1, 3]
print("Sorted Array:", insertion_sort(arr))
|
```

